

Machine Learning Interview Preparation

Top & Most Frequently Asked Questions with Detailed Answers

Tailored for ML / Core Developer roles at service companies (e.g. TCS, Infosys, Wipro, Accenture)

Covers: ML Fundamentals • Feature Engineering • Scikit-learn • Models • Model Evaluation

35+ Questions | Concept + Code + Interview Tips

Contents

1. **Section 1 — Machine Learning Fundamentals** (Q1-Q9)
2. **Section 2 — Feature Engineering** (Q10-Q17)
3. **Section 3 — Scikit-learn (Sklearn)** (Q18-Q24)
4. **Section 4 — ML Models & Algorithms** (Q25-Q33)
5. **Section 5 — Model Evaluation & Validation** (Q34-Q40)
6. **Quick Revision Cheat-Sheet**

How to use this guide: Read the answer, understand the intuition, and be ready to give a 2-3 sentence crisp version out loud. Interviewers value clarity over jargon. Where code is shown, be able to explain *what* each line does and *why*.

Section 1 — Machine Learning Fundamentals

Q1. What is Machine Learning? What are its main types?

Answer:

Machine Learning (ML) is a branch of Artificial Intelligence where systems learn patterns from data and improve at a task without being explicitly programmed with rules. Instead of writing logic by hand, we feed data to an algorithm that builds a mathematical model to make predictions or decisions.

The three main types are:

- **Supervised Learning:** The data has labelled outputs. The model learns the mapping from inputs (X) to a known output (y). Examples: spam detection (classification), house-price prediction (regression).
- **Unsupervised Learning:** The data has no labels. The model finds hidden structure or groupings. Examples: customer segmentation (clustering), dimensionality reduction (PCA).
- **Reinforcement Learning:** An agent learns by interacting with an environment and receiving rewards or penalties. Examples: game-playing agents, robotics.

Interview tip: Mention a 4th category increasingly common in practice — *semi-supervised learning* — where a small amount of labelled data is combined with a large amount of unlabelled data.

Q2. What is the difference between Supervised and Unsupervised learning?

Aspect	Supervised	Unsupervised
Data	Labelled (X and y)	Unlabelled (only X)
Goal	Predict an output	Discover structure / patterns
Tasks	Classification, Regression	Clustering, Association, Dim. reduction
Examples	Linear/Logistic Regression, Random Forest	K-Means, PCA, DBSCAN
Evaluation	Accuracy, RMSE, F1 (clear ground truth)	Harder — silhouette score, inertia

Q3. Explain the Bias-Variance Tradeoff.

Answer:

Total prediction error of a model can be decomposed into three parts: **bias**, **variance**, and irreducible noise.

- **Bias** is error from overly simple assumptions. High bias means the model underfits — it misses real patterns (e.g. fitting a straight line to curved data).
- **Variance** is error from sensitivity to small fluctuations in training data. High variance means the model overfits — it memorises noise and fails on new data.

The tradeoff: reducing bias (more complex model) usually increases variance, and vice-versa. The goal is the **sweet spot** that minimises total error on unseen data. Techniques like regularisation, cross-validation, and ensembling help balance the two.

Q4. What are Overfitting and Underfitting? How do you prevent overfitting?

Answer:

Overfitting occurs when a model learns the training data too well, including its noise, so it performs great on training data but poorly on test data (low bias, high variance). **Underfitting** is the opposite — the model is too simple to capture the underlying pattern, so it performs poorly on both training and test data (high bias).

Ways to prevent **overfitting**:

- Get more training data / use data augmentation.
- Apply regularisation (L1, L2, dropout for neural nets).
- Reduce model complexity (fewer features, shallower trees).
- Use cross-validation to tune hyperparameters.
- Early stopping (stop training when validation error rises).
- Ensemble methods (e.g. Random Forest averages many trees).
- Prune decision trees; set `max_depth`, `min_samples_leaf`.

Q5. What is the difference between Training, Validation, and Test sets?

Answer:

- **Training set:** Used to fit the model (learn the parameters).
- **Validation set:** Used to tune hyperparameters and pick the best model during development. The model never trains on it directly.
- **Test set:** Held out until the very end to give an unbiased estimate of real-world performance. Used only once.

A common split is 70/15/15 or 80/10/10. When data is limited, **k-fold cross-validation** replaces a fixed validation set.

Q6. What is Regularisation? Explain L1 vs L2.

Answer:

Regularisation adds a penalty term to the loss function to discourage large/complex coefficients, which reduces overfitting.

- **L1 (Lasso):** adds the sum of *absolute* values of coefficients ($\lambda \sum |w|$). It can shrink some coefficients exactly to zero, performing automatic **feature selection**.
- **L2 (Ridge):** adds the sum of *squared* coefficients ($\lambda \sum w^2$). It shrinks coefficients toward zero but rarely to exactly zero; it handles correlated features well.
- **Elastic Net:** combines both L1 and L2.

The hyperparameter λ (called `alpha` in sklearn) controls the strength: larger λ = more regularisation = simpler model.

Q7. What is the Curse of Dimensionality?

Answer:

As the number of features (dimensions) grows, the volume of the feature space grows exponentially, so data becomes sparse. Points become roughly equidistant, distance-based methods (KNN, K-Means) lose meaning, more data is needed to generalise, and overfitting risk rises. Remedies: feature selection, dimensionality reduction (PCA), and regularisation.

Q8. Parametric vs Non-Parametric models?

	Parametric	Non-Parametric
Assumption	Fixed form & fixed number of parameters	No fixed form; parameters grow with data
Examples	Linear/Logistic Regression, Naïve Bayes	KNN, Decision Trees, SVM (RBF)
Pros	Fast, needs less data, interpretable	Flexible, fits complex patterns
Cons	Can underfit if assumption wrong	Needs more data, slower, can overfit

Q9. What is the difference between Classification and Regression?

Answer:

Both are supervised tasks. **Classification** predicts a discrete category/label (e.g. spam vs not-spam, churn vs no-churn) and is evaluated with accuracy, precision, recall, F1, ROC-AUC. **Regression** predicts a continuous numeric value (e.g. price, temperature) and is evaluated with MSE, RMSE, MAE, R^2 .

Section 2 — Feature Engineering

Q10. What is Feature Engineering and why is it important?

Answer:

Feature engineering is the process of using domain knowledge to create, transform, and select input variables (features) so that the ML model can learn better. It includes handling missing values, encoding categoricals, scaling, creating interaction/derived features, and binning.

It is important because **the quality of features often matters more than the choice of algorithm** — good features can make a simple model outperform a complex model trained on raw data. A common saying: "Garbage in, garbage out."

Q11. How do you handle missing values?

Answer:

- **Remove:** drop rows/columns if missingness is small or a column is mostly empty.
- **Imputation:** fill with mean/median (numeric, median is robust to outliers), mode (categorical), or a constant/"Unknown".
- **Model-based:** KNN imputer, iterative imputation, or predicting the missing value.
- **Indicator flag:** add a binary column marking "was missing", since missingness itself can be informative.

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test) # use train statistics
```

Q12. How do you encode categorical variables?

Technique	When to use
Label Encoding	Ordinal data with natural order (Low < Medium < High), or tree models.
One-Hot Encoding	Nominal data with no order & low cardinality. Creates a binary column per category.
Target / Mean Encoding	High-cardinality categoricals; replace category with mean of target (watch for leakage).
Frequency Encoding	Replace category with its count/frequency.

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
encoded = ohe.fit_transform(X_train[['city']])
```

Tip: One-hot on high-cardinality columns explodes dimensionality — prefer target/frequency encoding there.

Q13. What is Feature Scaling? Normalisation vs Standardisation?

Answer:

Scaling brings features onto a comparable range so that no feature dominates due to its units. It is essential for distance- and gradient-based algorithms (KNN, SVM, K-Means, Logistic/Linear Regression, neural nets) but **not needed for tree-based models** (Decision Trees, Random Forest, XGBoost).

- **Normalisation (Min-Max):** rescales to [0, 1]. Formula: $(x - \min) / (\max - \min)$. Sensitive to outliers.
- **Standardisation (Z-score):** rescales to mean 0, std 1. Formula: $(x - \mu) / \sigma$. More robust; preferred default.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # fit only on train to avoid leakage
```

Q14. How do you handle imbalanced datasets?

Answer:

An imbalanced dataset (e.g. 99% non-fraud, 1% fraud) makes a model biased toward the majority class. Strategies:

- **Resampling:** oversample the minority (SMOTE), or undersample the majority.
- **Class weights:** set `class_weight='balanced'` so errors on the minority cost more.
- **Right metrics:** use Precision, Recall, F1, ROC-AUC / PR-AUC — never rely on plain accuracy.
- **Anomaly detection** framing for extreme imbalance.
- Collect more minority data if possible.

Q15. What are Feature Selection methods?

Answer:

- **Filter methods:** use statistics independent of the model — correlation, chi-square, ANOVA, mutual information.
- **Wrapper methods:** evaluate feature subsets using the model — Recursive Feature Elimination (RFE), forward/backward selection.
- **Embedded methods:** selection happens during training — Lasso (L1), tree feature importances.

Benefits: reduces overfitting, speeds up training, improves interpretability, fights the curse of dimensionality.

Q16. What is PCA (Principal Component Analysis)?

Answer:

PCA is an **unsupervised dimensionality-reduction** technique. It transforms correlated features into a smaller set of uncorrelated **principal components** that capture the maximum variance in the data. The first component captures the most variance, the second the next most (orthogonal to the first), and so on.

Use it to reduce dimensionality, remove multicollinearity, speed up training, and visualise high-dimensional data. **Always scale features before PCA.** Downside: components are linear combinations, so interpretability is lost.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=0.95) # keep 95% of variance
X_reduced = pca.fit_transform(X_scaled)
```

Q17. What is Data Leakage? How do you prevent it?

Answer:

Data leakage is when information from outside the training data (often from the test set or the future) sneaks into model training, giving unrealistically good validation scores that collapse in production. Classic example: fitting a scaler or imputer on the full dataset before splitting.

Prevent it by: splitting *before* any preprocessing, fitting transformers on train only (then `transform` on test), using `Pipeline` so all steps respect the split, and being careful with target-derived features and time-series ordering.

Section 3 — Scikit-learn (Sklearn)

Q18. What is Scikit-learn and what is its typical workflow?

Answer:

Scikit-learn is the most popular open-source Python library for classical (non-deep-learning) ML. It provides a consistent API for preprocessing, models, model selection, and metrics. Typical workflow:

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler().fit(X_train)
X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
preds = model.predict(X_test)
print(accuracy_score(y_test, preds))
```

Q19. Explain fit(), transform(), fit_transform() and predict().

Method	Purpose
<code>fit()</code>	Learns parameters from data (e.g. mean/std for a scaler, splits for a tree).
<code>transform()</code>	Applies the learned transformation to data (scaling, encoding).
<code>fit_transform()</code>	Convenience: fit then transform in one call — use on training data only.
<code>predict()</code>	Uses a fitted estimator to produce predictions on new data.

Key rule: Call `fit_transform` on the train set, but only `transform` on the test set — otherwise you leak test information.

Q20. What is a Scikit-learn Pipeline and why use it?

Answer:

A `Pipeline` chains preprocessing steps and a final estimator into one object. Benefits: cleaner code, prevents data leakage (transformers are fit only on training folds during cross-validation), and makes the model easy to deploy as a single unit.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression())
])
pipe.fit(X_train, y_train)
pipe.predict(X_test)
```

Q21. What is ColumnTransformer?

Answer:

It applies different preprocessing to different columns — e.g. scale numeric columns and one-hot encode categorical columns in a single step. It pairs naturally with `Pipeline`.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

pre = ColumnTransformer([
    ('num', StandardScaler(), ['age', 'income']),
    ('cat', OneHotEncoder(handle_unknown='ignore'), ['city'])
])
```

Q22. How do you tune hyperparameters? (GridSearchCV vs RandomizedSearchCV)

Answer:

Hyperparameters are set before training (e.g. tree depth, learning rate, C in SVM), unlike parameters which are learned. Tuning finds the best combination using cross-validation.

- **GridSearchCV:** exhaustively tries every combination in a grid. Thorough but expensive.
- **RandomizedSearchCV:** samples random combinations — faster, often nearly as good, better for large search spaces.

```
from sklearn.model_selection import GridSearchCV
params = {'n_estimators': [100, 200], 'max_depth': [5, 10, None]}
grid = GridSearchCV(RandomForestClassifier(), params, cv=5, scoring='f1')
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
```

Q23. What does `train_test_split` do, and what is stratify?

Answer:

`train_test_split` randomly partitions data into train and test subsets. `random_state` makes the split reproducible. `stratify=y` preserves the class proportions in both splits — important for imbalanced classification so the test set is representative.

Q24. Difference between `predict()` and `predict_proba()`?

Answer:

`predict()` returns the final class label (e.g. 0 or 1) using a default 0.5 threshold. `predict_proba()` returns the probability of each class, which lets you choose a custom threshold — useful for imbalanced problems or when false positives/negatives have different costs (e.g. medical or fraud screening).

Section 4 — ML Models & Algorithms

Q25. Explain Linear Regression.

Answer:

Linear Regression models a continuous target as a linear combination of features: $y = w_0 + w_1x_1 + \dots + w_nx_n$. It learns the weights by minimising the Mean Squared Error (often via Ordinary Least Squares or gradient descent). Key assumptions: linearity, independence of errors, homoscedasticity (constant variance), and normally distributed residuals. Simple, fast, and interpretable, but underfits non-linear data.

Q26. Explain Logistic Regression. Is it regression or classification?

Answer:

Despite the name, Logistic Regression is a **classification** algorithm. It applies the **sigmoid** function to a linear combination of features to output a probability between 0 and 1, then thresholds it (default 0.5) to assign a class. It is trained by minimising log-loss (cross-entropy). It is interpretable (coefficients relate to log-odds) and a strong baseline for binary classification. Use softmax for multi-class.

Q27. How does a Decision Tree work?

Answer:

A Decision Tree splits the data into branches based on feature thresholds, choosing splits that best separate the target. For classification it minimises **Gini impurity** or **entropy (information gain)**; for regression it minimises variance/MSE. It keeps splitting until a stopping rule (max depth, min samples) is met.

Pros: interpretable, handles non-linearity, no scaling needed, handles mixed data types. **Cons:** prone to overfitting (control with pruning, `max_depth`, `min_samples_leaf`), unstable to small data changes.

Q28. What is a Random Forest? Why is it better than a single tree?

Answer:

Random Forest is an **ensemble of decision trees** built with **bagging** (bootstrap aggregating). Each tree is trained on a random sample of rows and a random subset of features; predictions are averaged (regression) or voted (classification).

It beats a single tree because averaging many de-correlated trees **reduces variance** and overfitting, giving more robust, accurate predictions. It also provides feature-importance scores. Trade-off: less interpretable and slower than one tree.

Q29. Explain Bagging vs Boosting.

	Bagging	Boosting
Training	Trees built in parallel, independently	Trees built sequentially, each fixing prior errors
Goal	Reduce variance	Reduce bias (and variance)
Data	Random bootstrap samples	Reweights / focuses on misclassified points
Examples	Random Forest	AdaBoost, Gradient Boosting, XGBoost, LightGBM
Risk	Less prone to overfit	Can overfit if not tuned

Q30. What is Gradient Boosting / XGBoost?

Answer:

Gradient Boosting builds trees sequentially, where each new tree is trained to predict the **residual errors** (gradients of the loss) of the combined previous trees. **XGBoost** is an optimised, regularised implementation famous for winning many Kaggle competitions; it adds L1/L2 regularisation, handles missing values, supports parallelisation, and includes early stopping. Key hyperparameters: `learning_rate`, `n_estimators`, `max_depth`, `subsample`.

Q31. How does Support Vector Machine (SVM) work?

Answer:

SVM finds the **optimal hyperplane** that separates classes with the **maximum margin** (largest distance to the nearest points, called support vectors). For non-linear data it uses the **kernel trick** (e.g. RBF, polynomial) to map data into a higher-dimensional space where it becomes separable. The `c` parameter controls the margin-vs-misclassification trade-off. Works well in high dimensions but is slow on very large datasets and needs scaling.

Q32. Explain KNN and Naive Bayes.

Answer:

K-Nearest Neighbours (KNN): a lazy, instance-based algorithm. To classify a point it looks at the `k` closest training points (by distance) and takes a majority vote. No real "training" phase; needs feature scaling; slow at prediction time on big data; sensitive to the curse of dimensionality and the choice of `k`.

Naive Bayes: a probabilistic classifier based on Bayes' theorem with a "naive" assumption that features are conditionally independent

given the class. Despite the strong assumption, it is fast, works well on text/spam classification, and needs little data.

Q33. How does K-Means clustering work?

Answer:

K-Means is an unsupervised algorithm that partitions data into `k` clusters: (1) pick k initial centroids, (2) assign each point to the nearest centroid, (3) recompute centroids as the mean of assigned points, (4) repeat until convergence. Choose `k` using the **Elbow method** (plot inertia vs k) or the **silhouette score**. Needs scaling, assumes spherical clusters, and is sensitive to initialisation (use `k-means++`).

Section 5 — Model Evaluation & Validation

Q34. What is a Confusion Matrix?

Answer:

A confusion matrix is a table comparing predicted vs actual classes for a classifier:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

All key classification metrics are derived from these four numbers.

Q35. Explain Accuracy, Precision, Recall, and F1-Score.

Metric	Formula	Meaning
Accuracy	$(TP+TN) / \text{Total}$	Overall correctness. Misleading on imbalanced data.
Precision	$TP / (TP+FP)$	Of predicted positives, how many are correct. "Cost of false alarms."
Recall (Sensitivity)	$TP / (TP+FN)$	Of actual positives, how many we caught. "Cost of misses."
F1-Score	$2 \cdot (P \cdot R) / (P+R)$	Harmonic mean of precision & recall. Good for imbalance.

Precision vs Recall — when? Favour **recall** when missing a positive is costly (cancer detection, fraud). Favour **precision** when false alarms are costly (spam filter marking real mail as spam).

Q36. What is the ROC Curve and AUC?

Answer:

The **ROC curve** plots the True Positive Rate (recall) against the False Positive Rate across all classification thresholds. **AUC** (Area Under the Curve) summarises it as a single number: 0.5 = random guessing, 1.0 = perfect. AUC measures how well the model ranks/separates the classes regardless of threshold. For heavily imbalanced data, the **Precision-Recall AUC** is often more informative.

Q37. What metrics are used for Regression?

Metric	Notes
MAE (Mean Absolute Error)	Average absolute error; robust to outliers; same units as target.
MSE (Mean Squared Error)	Penalises large errors more (squares them); sensitive to outliers.
RMSE (Root MSE)	Square root of MSE; same units as target; common default.
R² (Coefficient of Determination)	Fraction of variance explained (1 = perfect, 0 = mean baseline, can be negative).

Q38. What is Cross-Validation? Explain K-Fold.

Answer:

Cross-validation gives a more reliable performance estimate than a single train/test split. In **k-fold CV**, the data is split into k equal folds; the model trains on k–1 folds and validates on the remaining fold, repeating k times so every fold is used once for validation. The scores are averaged.

Variants: **Stratified k-fold** (preserves class ratios — default for classification), **Leave-One-Out** (k = n), and **TimeSeriesSplit** (respects time order, no shuffling).

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='f1')
print(scores.mean(), scores.std())
```

Q39. How do you choose the right evaluation metric?

Answer:

It depends on the problem and business cost of errors:

- **Balanced classification:** Accuracy, ROC-AUC.
- **Imbalanced classification:** Precision, Recall, F1, PR-AUC — not accuracy.
- **Regression:** RMSE/MAE (and R² for explained variance).
- Always align the metric with what the business actually cares about (e.g. recall for fraud, precision for recommendations).

Q40. What steps would you follow in an end-to-end ML project?

Answer:

1. **Problem definition** — understand the goal and success metric.
2. **Data collection & understanding** (EDA) — explore distributions, correlations, missingness.
3. **Data cleaning & preprocessing** — handle missing values, outliers, types.
4. **Feature engineering & selection.**
5. **Train/test split** (and CV setup).
6. **Model selection & training** — start with a simple baseline.
7. **Hyperparameter tuning** with cross-validation.
8. **Evaluation** on the held-out test set with the right metric.
9. **Deployment** (API/batch) and **monitoring** for data drift & performance decay.

Quick Revision Cheat-Sheet

Algorithm picker

Need	Try
Interpretable baseline (classification)	Logistic Regression
Interpretable baseline (regression)	Linear Regression
Strong tabular performance, low effort	Random Forest
Top tabular accuracy	XGBoost / LightGBM
Small data, high dimensions	SVM
Text / spam classification	Naive Bayes
Unlabelled grouping	K-Means / DBSCAN
Reduce dimensions	PCA

When is scaling required?

Yes: KNN, SVM, K-Means, Logistic/Linear Regression (with gradient descent), PCA, Neural Networks.

No: Decision Tree, Random Forest, XGBoost and other tree-based models.

Metric quick-map

Classification (balanced): Accuracy, ROC-AUC | **Classification (imbalanced):** Precision, Recall, F1, PR-AUC | **Regression:** RMSE, MAE, R^2

30-second talking points

- **Overfitting?** → more data, regularisation, simpler model, CV, early stopping, ensembling.
- **Imbalance?** → SMOTE / class weights + F1/PR-AUC, not accuracy.
- **Leakage?** → split first, fit transformers on train only, use Pipelines.
- **Bias-variance?** → simple = high bias (underfit); complex = high variance (overfit); balance via CV.
- **Bagging vs Boosting?** → bagging cuts variance (parallel); boosting cuts bias (sequential).

Final advice: In interviews, structure answers as *Definition* → *Why it matters* → *Short example*. Be honest about trade-offs — admitting a model's weaknesses shows real understanding. Good luck!